

Acceso a Datos - Semana 3

Ficheros binarios y serialización en Java

Colección de ejercicios prácticos con enunciados desarrollados

Objetivo de la semana: practicar la lectura y escritura de datos binarios, el uso de `DataInputStream` y `DataOutputStream`, la serialización de objetos con `Serializable` y `ObjectInputStream/ObjectOutputStream`, y la gestión de colecciones persistentes.

Ejercicio 1. Registro binario de temperaturas

Una empresa de mantenimiento quiere almacenar las temperaturas registradas por un sensor durante una jornada de trabajo. Las temperaturas se han obtenido cada hora y deben conservarse en un fichero binario para poder analizarlas posteriormente.

Desarrolla un programa Java que cree un fichero llamado `temperaturas.dat` y almacene en él las siguientes temperaturas:

18.5

19.2

21.7

23.1

22.8

20.4

- La temperatura media.
- La temperatura máxima.
- La temperatura mínima.

Las temperaturas deberán escribirse utilizando un flujo de salida binario adecuado.

Posteriormente, el programa deberá abrir nuevamente el fichero, leer todos los valores almacenados y mostrarlos por pantalla. El programa deberá controlar correctamente el final del fichero y gestionar las posibles excepciones de entrada/salida.

Ejercicio 2. Fichero binario con datos de alumnos

Un centro educativo quiere guardar información básica de varios alumnos sin utilizar todavía una base de datos.

Cada alumno tendrá los siguientes datos:

Número de expediente

Nombre

Edad

Nota media

Realiza un programa Java que solicite al usuario los datos de 5 alumnos y los almacene en un fichero binario llamado alumnos.dat. Utiliza los métodos adecuados de DataOutputStream, teniendo en cuenta el tipo de cada dato.

Una vez almacenados los alumnos, el programa deberá abrir el fichero utilizando DataInputStream, leer todos los registros y mostrar por pantalla una salida similar a:

Expediente: 1024

Nombre: Ana García

Edad: 20

Nota media: 8.35

El programa deberá seguir leyendo alumnos hasta alcanzar el final del fichero.

Ejercicio 3. Productos de un almacén

Una pequeña empresa desea almacenar temporalmente la información de los productos de su almacén en un fichero binario.

De cada producto se conoce:

Código del producto

Nombre

Precio

Número de unidades disponibles

- El número total de productos diferentes.
- El valor total del almacén.

Desarrolla un programa que permita introducir productos desde teclado y almacenarlos en un fichero denominado productos.dat. El usuario podrá introducir tantos productos como desee. Para finalizar la entrada deberá introducir el código 0.

Después de cerrar el fichero, el programa deberá volver a abrirlo y mostrar todos los productos almacenados.

El valor de cada producto se obtiene mediante precio x unidades, y el valor total del almacén será la suma del valor de todos los productos.

Ejercicio 4. Buscar información dentro de un fichero binario

Partiendo del fichero productos.dat creado en el ejercicio anterior, realiza un nuevo programa que permita al usuario introducir el código de un producto.

El programa deberá recorrer el fichero binario buscando un producto cuyo código coincida con el indicado.

- Código.
- Nombre.
- Precio.
- Stock.

Si encuentra el producto, deberá mostrar sus datos. En caso contrario deberá informar al usuario mediante un mensaje indicando que no existe ningún producto con el código indicado.

La búsqueda deberá realizarse leyendo secuencialmente el fichero hasta encontrar el producto o llegar al final.

Ejercicio 5. Clase Persona serializable

Crea una clase denominada Persona que contenga los siguientes atributos:

String nombre;

String apellidos;

int edad;

String dni;

- Constructor.
- Métodos get y set.
- Método toString().

La clase deberá implementar la interfaz Serializable.

Posteriormente, crea un programa principal que genere tres objetos Persona y los guarde en un fichero denominado personas.dat utilizando ObjectOutputStream.

A continuación, abre el fichero mediante ObjectInputStream, recupera los objetos almacenados y muestra sus datos por pantalla. El objetivo es comprobar cómo Java permite almacenar un objeto completo sin tener que guardar cada atributo individualmente.

Ejercicio 6. Serialización de empleados

Una empresa quiere realizar una copia sencilla de los datos de sus trabajadores.

Crea una clase Empleado con los siguientes atributos:

```
int id;  
  
String nombre;  
  
String departamento;  
  
double salario;
```

- Todos los empleados.
- El número total de empleados.
- El salario medio.
- El empleado que tiene el salario más alto.

La clase deberá implementar Serializable. En el programa principal crea al menos 5 empleados y almacénalos dentro de un ArrayList<Empleado>.

Posteriormente, serializa directamente el ArrayList completo en un fichero empleados.dat. Una vez escrito el fichero, recupéralo y muestra la información solicitada.

Ejercicio 7. Catálogo de libros serializado

Una biblioteca quiere disponer de una pequeña aplicación para almacenar su catálogo de libros.

Crea una clase Libro con:

```
String isbn;  
  
String titulo;  
  
String autor;  
  
int anioPublicacion;  
  
boolean prestado;
```

Menú:

1. Añadir libro
2. Mostrar libros
3. Guardar catálogo
4. Cargar catálogo
0. Salir

Haz que la clase sea serializable. El programa deberá permitir introducir libros desde teclado y almacenarlos en un ArrayList<Libro>.

La opción Guardar catálogo deberá serializar la lista completa en biblioteca.dat. La opción Cargar catálogo deberá recuperar del fichero los datos guardados anteriormente.

Al cerrar y volver a ejecutar el programa, deberá ser posible recuperar el catálogo mediante la opción correspondiente.

Ejercicio 8. Gestión de vehículos

Un pequeño taller necesita almacenar información sobre los vehículos de sus clientes.

Crea una clase Vehiculo serializable con los siguientes atributos:

String matricula;

String marca;

String modelo;

int kilometros;

double costeReparacion;

- Todos los vehículos registrados.
- Los vehículos con más de 100000 kilómetros.
- El vehículo cuya reparación sea más cara.
- El importe total facturado en reparaciones.

El programa deberá permitir registrar varios vehículos y almacenarlos mediante serialización en el fichero vehiculos.dat. Después de recuperar los datos del fichero, deberá mostrar la información indicada.

Ejercicio 9. Guardar objetos relacionados

Una academia quiere almacenar los cursos que imparte.

Crea una clase Profesor con:

String nombre;

String email;

Y una clase Curso con:

String nombre;

int horas;

Profesor profesor;

Ambas clases deberán ser serializables. Crea varios objetos Profesor y Curso, relacionando cada curso con el profesor que lo imparte.

Serializa una lista de cursos en cursos.dat. Posteriormente, recupera los datos y muestra para cada curso su nombre, duración, profesor y correo electrónico.

El objetivo es comprobar que la serialización también conserva los objetos que están contenidos dentro de otros objetos.

Ejercicio 10. Atributos que no deben serializarse

Una aplicación guarda información de usuarios. Crea una clase Usuario con:

String nombre;

String email;

String password;

La clase deberá implementar Serializable. Sin embargo, por seguridad, la contraseña no deberá almacenarse en el fichero.

Investiga y utiliza la palabra reservada transient para evitar que este atributo se serialice.

Crea un usuario, guárdalo en un fichero y recupéralo posteriormente. Muestra por pantalla los valores recuperados y comprueba qué valor contiene el atributo password después de la deserialización.

Explica mediante un comentario en el código por qué ocurre este comportamiento.

Ejercicio 11. Copia de seguridad de clientes

Una pequeña empresa mantiene una lista de clientes en memoria durante la ejecución de su programa.

Cada cliente contiene:

Código

Nombre

Teléfono

Email

Saldo pendiente

Menú:

1. Añadir cliente
2. Mostrar clientes
3. Buscar cliente
4. Eliminar cliente
5. Guardar copia de seguridad
6. Recuperar copia de seguridad
0. Salir

Implementa una clase Cliente serializable y desarrolla un programa que permita gestionar una lista de clientes.

La opción Guardar copia de seguridad serializará la lista completa en clientes_backup.dat. La opción Recuperar copia de seguridad deberá reemplazar la lista actual por los clientes almacenados anteriormente.

Ejercicio 12. Proyecto final: gestión de alumnos mediante serialización

El departamento de informática de un instituto desea desarrollar una aplicación sencilla para gestionar la información académica de sus alumnos sin utilizar todavía una base de datos.

Crea una clase Alumno con los siguientes atributos:

```
int numeroExpediente;  
  
String nombre;  
  
String apellidos;  
  
String ciclo;  
  
double notaMedia;
```

Menú:

1. Añadir alumno
2. Mostrar todos los alumnos
3. Buscar alumno por expediente
4. Mostrar alumnos aprobados
5. Mostrar alumno con mayor nota media
6. Eliminar alumno

7. Guardar datos

8. Cargar datos

0. Salir

- Intentar abrir un fichero que todavía no existe.
- Introducir datos incorrectos por teclado.
- Buscar un alumno inexistente.
- Intentar eliminar un alumno cuyo expediente no existe.

La clase deberá implementar `Serializable`. La aplicación mantendrá los alumnos en un `ArrayList<Alumno>` y los datos deberán almacenarse mediante serialización en `alumnos_serializados.dat`.

Al seleccionar Guardar datos, se almacenará la colección completa. Al seleccionar Cargar datos, se recuperarán todos los alumnos que existían en el momento de realizar el guardado.

Además, el programa deberá controlar posibles errores como los indicados.

Propuesta de secuenciación

Ejercicios 1-4: ficheros binarios con `DataInputStream` y `DataOutputStream`.

Ejercicios 5-6: introducción a la serialización.

Ejercicios 7-10: práctica y ampliación.

Ejercicios 11-12: actividades integradoras y de mayor dificultad.